

MEDIATEK

MT8676 Yocto SPI

Version: 1.1
Release date: 2025-03-24

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-08-12	薛佳	正式版
1.1	2025-03-24	甘文超	增加 章节 1.3.2 节点配置

目录

版本记录 2

目录 3

图片目录 4

表格目录 4

1 SPI 5

1.1 概述 5

1.1.1 简单介绍 5

1.1.2 名词解释 5

1.2 架构/流程概述 6

1.2.1 SPI 介绍 6

1.2.2 MT8676 SPI 特征 6

1.2.3 SPI 传输格式 8

1.2.4 接口说明 8

1.2.5 编程指南 9

1.3 配置/客制化指南 10

1.3.1 设备树 10

1.3.2 节点配置 11

1.3.3 PAD_SEL 11

1.3.4 频率 12

1.3.5 SPI 模式 12

1.3.6 FIFO 和 DMA 模式 12

1.3.7 支持多个设备 13

1.3.8 测试和调试 13

1.4 常见问题/故障排查 14

1.4.1 SPI 问题调试方法 14

1.4.2 dts 配置 14

1.4.3 确认 GPIO 模式 14

1.4.4 测量波形 15

1.4.5 如何打印 SPI 寄存器信息 15

1.4.6 寻求 MTK 帮助 15

附件一 附加条款..... 16

图片目录

图 1-1. SPI Master 和 SPI Slave 之间的引脚连接 6

图 1-2. 四种通信模式波形..... 7

图 1-3. 主设备与多设备引脚连接..... 7

图 1-4. SPI 传输格式..... 8

表格目录

表 1-1. 名词解释..... 5

1 SPI

1.1 概述

1.1.1 简单介绍

本章节介绍 MT8676 SPI 控制器的硬件、软件及其功能。

1.1.2 名词解释

表 1-1. 名词解释

缩写	解释
CPHA	Clock Phase 时钟相位
CPOL	Clock Polarity 时钟极性
CS	Chip Select 芯片选择引脚
DMA	Direct Memory Access 直接内存访问
FIFO	First In First Out 先进先出
MISO	Master In Slave Out, SPI Master 输入数据和 SPI Slave 输出数据
MOSI	Master Out Slave In, SPI Master 输出数据和 SPI Slave 输入数据
PIO	Programmed Input/Output Model 编程输入/输出模型
SCLK	SPI Clock, SPI Master (SPI Controller) 与 SPI Slave 数据传输频率
SPI	Serial Peripheral Interface 串行外设接口

1.2 架构/流程概述



图 1-1. SPI Master 和 SPI Slave 之间的引脚连接

1.2.1 SPI 介绍

SPI 接口是一种串行、四引脚传输协议。图 1-1 是 SPI Master 与 SPI Slave 之间的连接示例。SPI 控制器接口是一个主机，负责与从机之间的数据传输。通常 SPI 会接的设备有：闪存、触控板等。

1.2.2 MT8676 SPI 特征

- 提供 8 个 SPI 端口
- 支持 DMA 和 FIFO 模式两种传输模式
- FIFO 模式在一次传输中最多支持 32 个字节
- 如果传输长度小于 1024 字节，则 DMA 模式最多支持 1024 字节
- DMA 模式支持 1024 字节的倍数（长度 = 循环次数 * 1024， $1 \leq \text{循环次数} \leq 256$ ）
- 最大传输 频率为 52 MHz
- 有四种通信模式可用（模式 0、1、2、3），参见图 1-2
- 这基本上定义了 MOSI 线路切换的 SCLK 边沿、主机对 MISO 线路进行采样的 SCLK 边沿和 SCLK 信号稳定电平（即时钟电平，当时钟未激活时，高电平或低电平）。每种模式都由一对称为“时钟极性”（CPOL）和“时钟相位”（CPHA）的参数定义

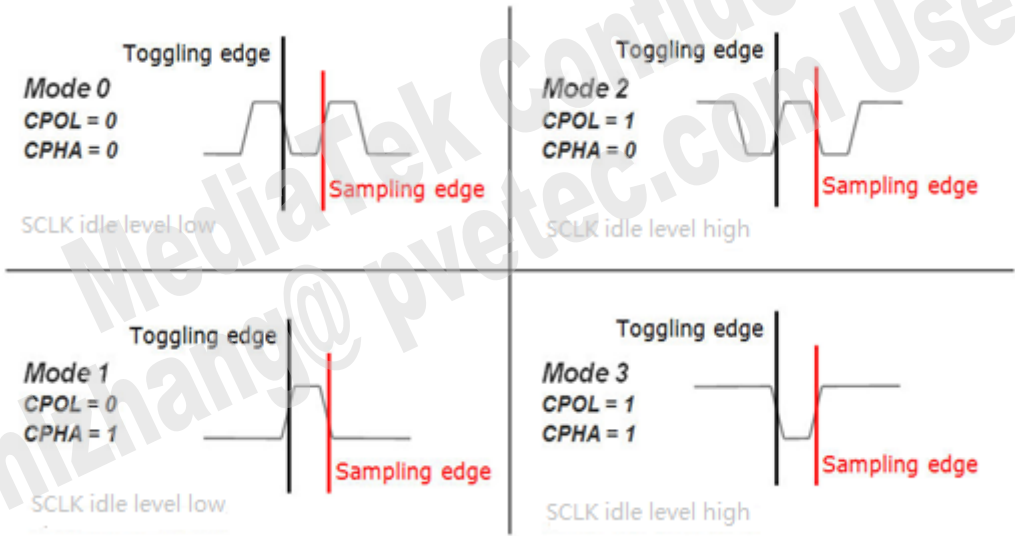


图 1-2. 四种通信模式波形

- SPI 控制器只有一个 CS 引脚，也就是说，它只能支持单从。但是，您可以将 GPIO 用作 CS 来支持多个设备。如图 1-3 所示。

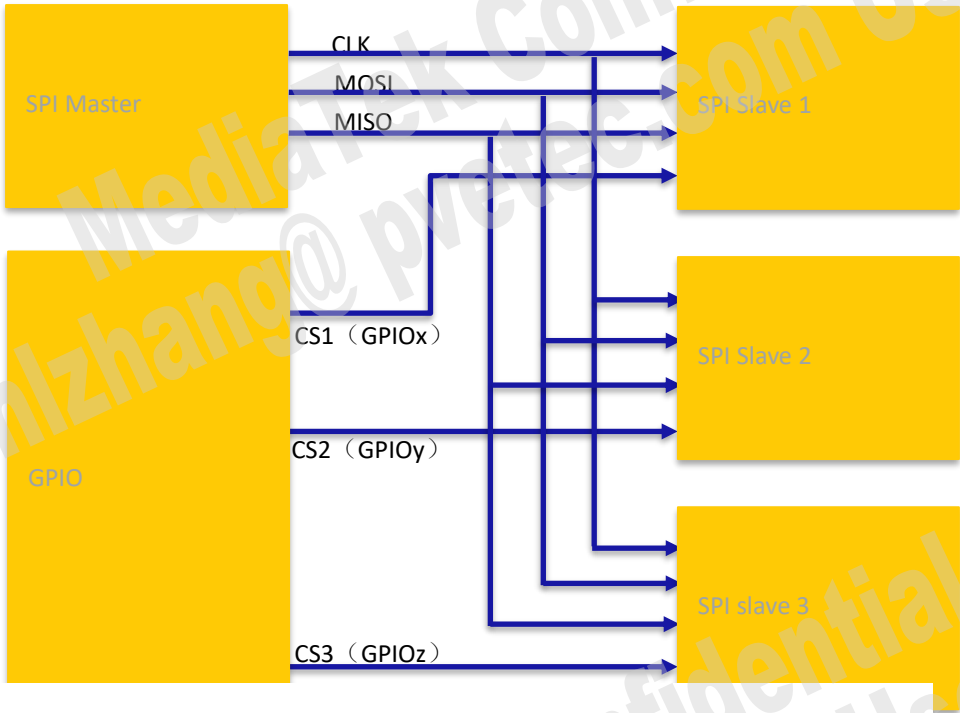


图 1-3. 主设备与多设备引脚连接

1.2.3 SPI 传输格式

在 DMA 模式下，要传输的数据应提前在系统中准备好。在 PIO 模式下，系统应首先推送传输到 SPI TX FIFO 的数据。收到 START 命令后，SPI 将连续向 Slave 发送数据，同时从 Slave 接收数据。

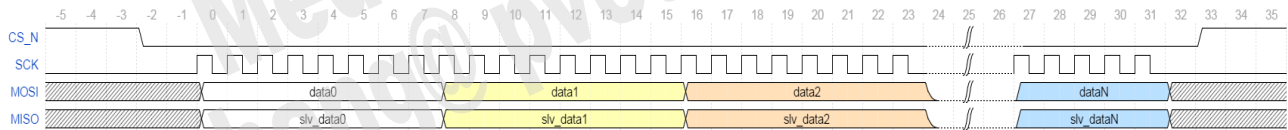


图 1-4. SPI 传输格式

1.2.4 接口说明

1.2.4.1 spi_sync

原型	参数	返回值
int spi_sync (struct spi_device *spi, struct spi_message *message)	SPI: 结构体 spi_device 的指针	成功返回 0，否则返回错误码
	Message：结构体 spi_message 的指针	

1.2.4.2 spi_async

原型	参数	返回值
int spi_async (struct spi_device *spi, struct spi_message *message)	SPI: 结构体 spi_device 的指针	成功返回 0，否则返回错误码
	Message: 结构体 spi_message 的指针	

1.2.4.3 spi_write_then_read

此例程的参数始终使用小型缓冲区进行复制，不应将其用于超过 32 个字节。性能敏感型或批量传输代码应改为使用带有 dma 安全缓冲区的 spi_{async, sync} () 调用。

原型	参数	返回值
int spi_write_then_read (struct spi_device *spi, const void *txbuf, unsigned n_tx)	SPI: 结构体 spi_device 的指针	成功返回 0，否则返回错误码
	txbuf: 要写入的数据	
	n_tx: txbuf 的大小 (以字节为单位)	

原型	参数	返回值
n_tx, unsigned n_tx, void *rxbuf, unsigned n_rx)	rxbuf: 数据将被读到的缓冲区	
	n_rx: rxbuf 的大小 (以字节为单位)	

1.2.5 编程指南

1.2.5.1 以 spi_sync 传输数据

数据在 SPI 控制器和 SPI 从设备之间交换，因此发送数据的步骤与接收数据的步骤相同。

例如，发送和接收数据：

```
struct spi_message msg;
struct spi_transfer xfer;

spi_message_init (&msg) ;
memset (&xfer, 0, sizeof (xfer) ) ;

xfer.tx_buf = local_tx_buf;
xfer.rx_buf = local_rx_buf;
xfer.len = data_length;
spi_message_add_tail (&xfer, &msg) ;

ret = spi_sync (spi, &msg) ;
if (ret) {
    .....;
}
```

1.2.5.2 以 spi_async 传输数据

函数 spi_sync 是同步的。如果客户想使用异步函数，请改用 spi_async。请注意，回调函数应注册到 spi_message.complete。

源代码路径：kernel-4.14/drivers/spi/spi.c.

```
spi_message.complete callback function:

static void xxxxxx_complete(void *args)
{
    //transfer was success.....;
}

Sending and receiving data in asynchronous:
```

```

struct spi_message msg;
struct spi_transfer xfer;

spi_message_init(&msg);
memset(&xfer, 0, sizeof(xfer));

xfer.tx_buf = local_tx_buf;
xfer.rx_buf = local_rx_buf;
xfer.len = data_length;
spi_message_add_tail(&xfer, &msg);

msg.complete = xxxxxx_complete;
msg.context = args;

ret = spi_async(spi, &msg);
if(ret) {
    .....;
}

```

1.3 配置/客制化指南

1.3.1 设备树

源代码路径: kernel-4.14/arch/arm64/boot/dts/mediatek/

- 添加 pinctrl 以设置 SPI pinmux

```

&pio {

    spi_pins: spi@0 {
        pins_spi {

            pinmux = <PINMUX_GPIOxxx_FUNC_SPIM1_CSB>,

                < PINMUX_GPIOxxx_FUNC_SPIM1_CLK >,

                < PINMUX_GPIOxxx_FUNC_SPIM1_MO >,

                < PINMUX_GPIOxxx_FUNC_SPIM1_MI >;

            bias-disable;

        };
    };
};

```

- 在 SPI 总线上添加从设备

```

&spi {

    pinctrl-names = "default";
    pinctrl-0 = <&spi_pins>;
    status = "okay";
    spidev0: spi@0 {
        compatible = "mediatek,spi-mt65xx-test";
    };
};

```

```

        reg = <0>;
        spi-max-frequency = <1000000>;
    };
};

```

1.3.2 节点配置

设备树中 SPI 节点默认是 “disabled” 状态: status = "disabled":

```

spi: spi@xxxxxxxx {
    compatible = "mediatek,mtxxxx-spi";
    mediatek,pad-select = <0>;
    .....
    .....
    status = "disabled";//默认关闭
};

```

使用时，在对应 project dts 中进行配置: status = "okay":

```

&spi {
    pinctrl-names = "default";
    pinctrl-0 = <&spi_pins>;
    mediatek,pad-select = <0>;
    status = "okay";//手动配置打开
    spidev0: spi@0 {
        compatible = "mediatek,spi-mt65xx-test";
        reg = <0>;
        spi-max-frequency = <1000000>;
    };
};

```

1.3.3 PAD_SEL

在某些 IC 上，一组 SPI 会有不同的 GPIO 可以选择，根据 GPIO 表格信息确定 SPI 使用哪组 GPIO。比如在 MT8675 上 SPI4 有 SPI4_A/SPI4_B/SPI4_C。

此时需要在 dts 中添加 mediatek,pad-select = <x>;

```

&spi {
    pinctrl-names = "default";
    pinctrl-0 = <&spi_pins>;
    mediatek,pad-select = <0>; //x = 0 1 2 分别对应 A B C。
    status = "okay";
    spidev0: spi@0 {
        compatible = "mediatek,spi-mt65xx-test";
        reg = <0>;
        spi-max-frequency = <1000000>;
    };
};

```

};

1.3.4 频率

SPI 控制器支持偶数倍分频，支持的最大频率 SPI 控制器为 52 MHz， $SPI_SCK \leq 52MHz$ 。

您可以通过两种方式设置 SPI_CLK：

1. 在设备树中，配置属性 `spi-max-frequency`，例如，`spi-max-frequency = <1000000>`；设置默认 SPI_CLK = 1MHz。
2. 在设备驱动中，配置结构 `spi_transfer` 的字段 `speed_hz`，例如 `xfer->speed_hz = 1000000`；设置 SPI_CLK = 1MHz，它将在此次传输生效。

1.3.5 SPI 模式

您可以通过两种方式设置 SPI 模式：

1. 在设备树中设置 SPI 模式

```
&spi {
    pinctrl-names = "default";
    pinctrl-0 = <&spi_pins>;
    status = "okay";
    spidev0: spi@0 {
        compatible = "mediatek,spi-mt65xx-test";
        reg = <0>;
        spi-cpol = <0>;
        spi-cpha = <0>;
        spi-max-frequency = <1000000>;
    };
};
```

2. 在设备驱动程序中设置 SPI 模式

```
static int xxxxxx_probe(struct spi_device *spi)
{
    .....;
    spi->mode = SPI_MODE_0; // SPI_MODE_0, SPI_MODE_1, SPI_MODE_2, SPI_MODE_3
    .....;
}
```

1.3.6 FIFO 和 DMA 模式

SPI 控制器支持 DMA 模式和 FIFO 模式传输数据。控制器通过传输数据长度自动选择。FIFO 模式的长度小于或等于 32 字节，DMA 模式的长度大于 32 字节。

1.3.7 支持多个设备

如果要在 SPI 总线上添加多个设备，可以使用 GPIO 作为支持 SPI_CS。例如，在 SPI 总线 0 上添加两个 SPI device，使用 GPIO136 和 GPIO30 作为 SPI_CS1 和 SPI_CS2，如下所示（GPIO 编号仅作示例使用，请以实际为准）：

```
&pio {
    spi_pins: spi@0 {
    pins_spi {
        pinmux = //<PINMUX_GPIO136_FUNC_SPIM1_CSB>, //SPI native cs
        < PINMUX_GPIO137_FUNC_SPIM1_CLK >,
        < PINMUX_GPIO138_FUNC_SPIM1_MO >,
        < PINMUX_GPIO139_FUNC_SPIM1_MI >;
        bias-disable;
    };
    pins_spi_cs {
        pinmux = < PINMUX_GPIO136_FUNC_SPIM1_CSB >, //SPI_CS1
        < PINMUX_GPIO30_FUNC_GPIO30 >, //SPI_CS2
        output-high;
    };
    };
};

&spi {
    pinctrl-names = "default";
    pinctrl-0 = <&spi_pins>;
    cs-gpios = <&pio 136 0>,<&pio 30 0>;
    status = "okay";
    spidev0: spi@0 {
        compatible = "xxxxxx";
        reg = <0>;
        spi-max-frequency = <1000000>;
    };
    spidev1: spi@1 {
        compatible = "xxxxxx";
        reg = <1>;
        spi-max-frequency = <1000000>;
    };
};
```

1.3.8 测试和调试

您可以按照上面介绍的步骤配置 SPI 后验证数据传输功能是否正常，例如验证 SPI 总线 0：

将 SPI_MOSI 连接到硬件平台上的 SPI_MISO，将 spi-mt65xx-dev.c 作设备驱动。

输入下列测试命令：

```
echo -w len=32 > /sys/ bus/spi/devices/spixxx/spi
```

```
echo -w len=1024 > /sys/ bus/spi/devices/spixxx/spi
```

spixxx 代表不同组的 SPI，以实际显示为准。

1.4 常见问题/故障排查

1.4.1 SPI 问题调试方法

大多数遇到的 SPI 问题都可先尝试通过如下方式调试：

- 检查 dts 配置、GPIO 模式、pad-select 属性是否正确
- 检查 SPI 寄存器信息是否正确
- 测量波形是否符合预期，比如 TX 数据是否正确打出、电压值是否正常
- 若收数据发生异常且比较有规律，比如移位，可以尝试降低速度或调整 tick_dly 的值
- 设备驱动代码写法是否标准，比如 TX、RX 缓冲区是否分配，TX、RX 数据长度是否一样

1.4.2 dts 配置

请按照前文的提供的信息，检查 dts 配置是否正确

1.4.3 确认 GPIO 模式

内核里查看引脚的状态

输入以下命令：

```
# cd /sys
# find -name mt_gpio
# cat mt_gpio //
```

要输入绝对路径

e.g.,

```
# cat /sys/devices/platform/soc/1000b000.pinctrl/mt_gpio
PIN: [MODE] [DIR] [DOUT] [DIN] [PULL_EN] [PULL_SEL] [IES] [SMT] [DRIVE] ( [R1] [R0] )
0: 0 0 0 0 1 0 1 0 0
1: 0 0 0 0 1 0 1 0 0
2: 0 1 1 1 1 0 1 0 0
3: 6 0 0 0 1 0 1 0 0
```

1.4.4 测量波形

使用示波器测量 SPI 波形是否符合预期。

1.4.5 如何打印 SPI 寄存器信息

在 driver/spi/spi-mt65xx.c 中添加如下内容

```
#define LOG_CLOSE 0
#define LOG_OPEN 1
u8 spi_log_status = LOG_OPEN;

#define spi_debug(fmt, args...) do { \
    if (spi_log_status == LOG_OPEN) {\
        pr_info("[spi]%s() " fmt, __func__, ##args);\
    } \
} while (0)

static void spi_dump_reg(struct mtk_spi *mdata)
{
    spi_debug("||*****%s*****||\n", __func__);
    spi_debug("cfg0:0x%.8x\n", readl(mdata->base + SPI_CFG0_REG));
    spi_debug("cfg1:0x%.8x\n", readl(mdata->base + SPI_CFG1_REG));
    spi_debug("cfg2:0x%.8x\n", readl(mdata->base + SPI_CFG2_REG));
    spi_debug("cmd :0x%.8x\n", readl(mdata->base + SPI_CMD_REG));
    spi_debug("tx_s:0x%.8x\n", readl(mdata->base + SPI_TX_SRC_REG));
    spi_debug("rx_d:0x%.8x\n", readl(mdata->base + SPI_RX_DST_REG));
    spi_debug("status1:0x%.8x\n", readl(mdata->base + SPI_STATUS1_REG));
    spi_debug("pad_sel:0x%.8x\n", readl(mdata->base + SPI_PAD_SEL_REG));
    spi_debug("||*****%s end*****||\n", __func__);
}
```

分别在 mtk_spi_fifo_transfer() and mtk_spi_dma_transfer() 两个函数中调用

mtk_spi_enable_transfer() 之前调用 spi_dump_reg()。

1.4.6 寻求 MTK 帮助

若经过前文的排查仍无法解决您的问题，向 MTK 寻求帮助时顺便提供前文提到的包含 SPI 寄存器信息的相关日志、波形图、dts、cat mt_gpio 等信息。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。